

A Deployable Platform for Real-World Network Traffic Classification Edge Devices

*Ankit Tiwari, *Sean Sullivan, †Charles Stolz, †Jielun Zhang, *Fuhao Li, *Enoch Hwang

* Computer Science Department, La Sierra University, Riverside, CA, 92505, USA

† School of Electrical Engineering & Computer Science, University of North Dakota, Grand Forks, ND, 58201, USA

Emails: {atiw547, ssul424, fli, ehwang}@lasierra.edu, {charles.stolz, jielun.zhang}@UND.edu

Abstract—Network Traffic Classification (NTC) is fundamental to network management, service differentiation, and performance optimization. However, most existing studies rely on offline datasets or controlled simulations, limiting their applicability to real operational environments. This paper presents a deployable end-to-end NTC platform designed for continuous monitoring of live network traffic. The system operates as an edge-side passive probe that captures Ethernet traffic non-intrusively using TShark and extracts 28-dimensional statistical flow features within sliding observation windows. To support real-time classification, we introduce a Per-Feature Normalized Mean Square Error (PF-NMSE) metric that normalizes feature-wise deviations between live samples and a pre-built application feature library. Each incoming flow is categorized by identifying the library entry with the smallest PF-NMSE score. Experiments on real network sessions demonstrate that the proposed PF-NMSE-based classifier accurately recognizes application types and streaming resolutions under varying bandwidth conditions, achieving an average accuracy of 97%. The platform can be directly deployed in operational networks and serves as a practical solution for continuous traffic monitoring and application-level behavior analysis.

Index Terms—Network Traffic Classification, Edge Computing, Statistical Feature Analysis, Video Streaming Identification, Real-World Deployment

I. INTRODUCTION

Network Traffic Classification (NTC) is a fundamental technique for network management, Quality of Service (QoS) optimization, and application-aware analytics [1]–[3]. By identifying the application types that generate network flows, operators can allocate bandwidth efficiently, enforce security policies, and monitor user behavior without inspecting sensitive payloads [4]–[6]. It also serves as an essential component in modern network intelligence systems, supporting adaptive routing, anomaly detection, and user experience optimization [7]–[10]. However, with the increasing adoption of encryption and the growing diversity of network environments, achieving accurate and interpretable traffic classification under real operational conditions remains a challenging task.

Traditional network traffic classification methods primarily rely on port-based identification and deep packet inspection (DPI) [11]–[13]. Port-based methods infer application types from predefined port assignments, whereas DPI analyzes packet payloads to extract protocol signatures. These approaches worked well in earlier networking environments,

but their applicability has become more limited as modern applications increasingly use encryption, dynamic port allocations. To address the inability to access payload contents in encrypted traffic, later studies introduced statistical machine learning approaches. These methods typically use hand-crafted flow statistics such as packet size distribution, inter-arrival time, and byte volume, and apply traditional models including support vector machines, decision trees, or k-nearest neighbors. For instance, the authors in [14] proposed an improved SVM model for network traffic classification. They utilized a filter-wrapper hybrid feature selection method and an improved grid search algorithm. They achieved a multi-class accuracy of 91.96%. The authors in [15] proposed a framework for classifying network traffic as VPN or non-VPN. They utilized time-related features and compared six machine-learning models, which achieved the highest average accuracy with the Gradient Boosting Tree model at 94.6%. With the advancement of deep learning, researchers further proposed neural-network-based traffic classification methods, such as multilayer perceptrons, convolutional neural networks, etc. [16]–[20]. For instance, in [17], the authors proposed a deep learning based packet classification by combining the CNN and RNN for IoT traffic. Their method gives the accurate rate above 96%. Moreover, the authors in [18] proposed an application-based network traffic classifier with MLP model to classify the data traffic. In [19], the author proposed a IoT traffic classification mechanism based on capsule network for smart cities. Ten-class aggressive traffic and ten-class normal traffic are classified and the average accuracy reach above 99%. However, most existing studies evaluate these methods on pre-collected and curated datasets, which lack the noise, bandwidth fluctuation, and multi-device interference common in real deployment environments [21]–[23]. As a result, their reported performance does not necessarily translate to real-world settings, and it remains unclear how these models behave when applied directly to live network traffic.

In this paper, we propose a deployable NTC platform that analyzes live Ethernet traffic in real environments. The system operates on an edge monitoring device connected to a mirrored/SPAN interface, enabling passive traffic capture without accessing packet payloads. The platform consists of two components: feature library construction and real-time classification. During feature library construction, the system collects traffic while a single application is active. Mixed

packets are captured with TShark and filtered using external IP addresses associated with the target application. Each n -second window is converted into a 28-dimensional statistical feature vector, and all labeled windows form a reference feature library representing the characteristic patterns of each application. During real-time analysis, the same feature extraction procedure is applied to incoming traffic. Since real network features often display heterogeneous scales and random fluctuations, direct comparison of raw feature differences is unstable. To address this, we introduce PF-NMSE (Per-Feature Normalized Mean Square Error), which compares the relative change ratio along each feature dimension and provides a more balanced and stable similarity measure. For each window, PF-NMSE is computed against all reference samples, and the closest match is identified. A confidence threshold τ ensures reliability: a prediction is reported only when the minimum PF-NMSE is below τ , otherwise the window is discarded. Experiments on real network sessions under varying bandwidth conditions show that the proposed system can accurately distinguish between application types and streaming resolutions, achieving an average classification accuracy of 97%. The main contributions of this work are summarized as follows: (i) We design and implement a deployable NTC platform capable of operating directly on live Ethernet traffic; (ii) We introduce PF-NMSE, a lightweight per-feature similarity measure that evaluates the relative change of each statistical feature, and combine it with a confidence threshold τ to ensure consistent and reliable matching under real network conditions. (iii) We evaluate the platform using traffic collected from several major video streaming services in real-world environments, and demonstrate that the system achieves an average accuracy of 97% with stable performance during live deployment.

The rest of the paper is organized as follows. Section II presents the proposed method. Section III details experiments and results. Section IV discusses the results of this paper. Section V concludes the paper.

II. METHODOLOGY

A. Overview of the Proposed Framework

The proposed NTC framework is a deployable edge-based system for real-time application identification. It passively monitors Ethernet traffic through mirrored or SPAN interfaces without accessing packet payloads, thereby ensuring both privacy protection and practical deployability. The overall framework consists of two main components: feature library construction and real-time analysis. In the feature library construction stage, the system collects network traffic while a single application, e.g., YouTube, Twitch, etc. is active. Ethernet packets are captured using TShark, and background traffic is filtered based on external IP addresses associated with the target application. Each observation window of n seconds is represented by d statistical features, such as packet count, mean packet size, total bytes, throughput, and mean inter-arrival time. The labeled samples are stored to build a reference feature library that characterizes the statistical

patterns of different applications. In the real-time analysis stage, the system continuously extracts the same set of features from live Ethernet traffic. For each observation window, the feature vector is compared with samples in the reference library using the NMSE metric, which ensures consistent comparison under bandwidth fluctuations and varying traffic intensity in real-time data. The label of the sample with the smallest distance is then assigned as the predicted result. A confidence threshold τ is also introduced to filter out low-confidence results, ensuring that only reliable predictions are reported. This mechanism enables the system to maintain accurate and stable classification performance under dynamic network conditions.

B. Feature Representation and Library Construction

The proposed framework begins with the construction of a feature library that provides the statistical foundation for real-time classification. During data collection, only one application is active at a time to ensure traffic purity. Ethernet packets are captured using TShark, recording metadata such as timestamp, source and destination IP addresses, and packet length. To minimize background interference, the system identifies external IP addresses associated with the active application and retains packets linked to those connections.

The captured traffic is divided into consecutive fixed-length observation windows of n seconds, denoted as $W_t = [t, t+n)$. For each window, all packets are aggregated to form a d -dimensional statistical feature vector $\mathbf{x}_t = [f_1, f_2, \dots, f_d] \in \mathbb{R}^d$, where each feature f_k represents a measurable traffic attribute such as packet count, mean packet size, total bytes, throughput, or mean inter-arrival time.

Each feature vector is paired with its corresponding application label y_t to form the feature library $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, which characterizes the statistical patterns of different applications (e.g., YouTube, Twitch, Amazon Prime Video, etc.). This library serves as a reference knowledge base during deployment and can be dynamically expanded as new traffic patterns are observed. The details is summarized in Alg. 1.

Algorithm 1: Reference Library Construction

Input : Traffic categories $\{1, \dots, M\}$; window size n

Output: Reference library $\mathcal{D} = \{d_1, d_2, \dots, d_M\}$

Initialize $\mathcal{D} = \emptyset$;

foreach category $m = 1 \dots M$ **do**

1. Capture Ethernet traffic via mirrored/SPAN port while only category m is active;
2. Filter flows based on external IPs associated with category m ;
3. Segment traffic into n -second windows;
4. Extract p -dimensional statistical features from each window;
5. Store all labeled feature vectors as subset d_m ;
6. Update library: $\mathcal{D} = \mathcal{D} \cup \{d_m\}$;

end

C. PF-NMSE Based Feature Comparison and Confidence Thresholding

During online operation, the system monitors live Ethernet traffic and extracts the same d -dimensional statistical feature vector for each observation window. To determine the most likely application category, the system compares the real-time vector $\mathbf{x}_t$ with every reference vector $\mathbf{x}_i \in \mathcal{D}$ using the proposed PF-NMSE metric. For each reference sample, the PF-NMSE is computed as

$$D_i = \frac{1}{d} \sum_{j=1}^d \frac{(x_{t,j} - x_{i,j})^2}{\max(x_{t,j}^2, x_{i,j}^2) + \epsilon}, \quad i = 1, \dots, N. \quad (1)$$

This formulation evaluates the relative per-feature deviation and prevents large-magnitude dimensions from dominating the comparison. The system then identifies the smallest distance $D_{\min}$ and uses the label of the corresponding reference sample as the predicted application. A confidence threshold τ is applied: only windows with $D_{\min} < \tau$ are accepted as valid predictions, while others are filtered as low-confidence to maintain reliable outputs under dynamic network conditions.

To ensure stability in dynamic network conditions, a distance threshold τ is introduced. Only when $D_{\min} < \tau$ is the classification considered valid and forwarded to the monitoring module; otherwise, the result is discarded. This rule can be formulated as

$$\delta_t = \begin{cases} 1, & D_{\min} < \tau, \\ 0, & D_{\min} \geq \tau, \end{cases} \quad (2)$$

where δ_t indicates whether the classification result for window W_t is accepted. The final output is expressed as $\mathcal{O}_t = (\hat{y}, D_{\min}, \delta_t)$, representing the predicted label, distance score, and decision flag. All accepted results ($\delta_t = 1$) are sequentially transmitted to the monitoring module, forming a continuous record of recognized application activities. This NMSE-based comparison and thresholding mechanism allows the system to maintain robust and consistent classification performance under real-world network variability. By tuning τ , the framework achieves a balance between sensitivity and reliability, ensuring stable operation across diverse traffic conditions. The algorithm is summarized in Alg. 2.

III. EVALUATION RESULT

A. Real-world Data collection

All experiments in this study are conducted on a network traffic dataset collected by our own monitoring platform. To reflect real-world deployment conditions and practical usage scenarios, we rely on traffic captured from a live home network rather than publicly available datasets. The dataset is collected using a dedicated edge monitoring device connected to the router's SPAN port, which passively captures all inbound and outbound Ethernet traffic. During each recording session, only one target application is active, ensuring that the captured traffic is exclusively generated by the selected application. Background traffic is removed based on application-specific external IP addresses, resulting in clean application-level flows

Algorithm 2: Real-time Network Traffic Classification

Input : Live Ethernet stream $\mathcal{T}$; reference library $\mathcal{D}$;
window size n ; threshold τ

Output: Predicted label $\hat{y}_t$ for each valid window

```

while external IP traffic is detected do
  1. Capture packets within an  $n$ -second window to
    form  $\mathbf{X}_t$ ;
  2. Extract  $p$ -dimensional feature vector  $\mathbf{x}_t$ ;
  3. Compute PF-NMSE distances  $D_i$  between  $\mathbf{x}_t$ 
    and all  $\mathbf{x}_i \in \mathcal{D}$  using Eq. (1);
  4. Identify minimal distance:  $D_{\min} = \min_i D_i$ ;
  if  $D_{\min} < \tau$  then
    Output predicted label:
    
$$\hat{y}_t = \text{label}(\arg \min_i D_i);$$

  else
    Exclude window  $\mathbf{X}_t$  to maintain reliable
    predictions;
  end
end

```

for feature extraction. In total, the dataset contains six application categories. Several widely used video streaming services are included as representative real-world applications for evaluation. The collected categories cover YouTube (YT), Twitch (TW), and Bilibili (BLBL) under three quality levels, i.e., high, mid, and low, which are labeled as YT-High, YT-Mid, YT-Low, TW-High, TW-Mid, TW-Low, BLBL-High, BLBL-Mid, and BLBL-Low. In addition, Amazon Prime Video (AP) and Netflix (NF) are treated as single categories due to their stable adaptive bitrate behavior, while TikTok Live (TT-Live) is included as a dedicated live-streaming category. These applications are selected to reflect diverse traffic dynamics commonly observed in operational networks. All statistical features are extracted online using TShark, which computes 28 per-window metrics during packet capture. Each recording is segmented into fixed 20-second observation windows, and a 28-dimensional statistical feature vector is generated for each window, including packet count, mean packet length, throughput, total bytes, and related flow statistics. A complete list of extracted features is provided in Table I.

B. Environment setup

In our experimental environment, we deployed two end devices connected together through a passive network tap to construct the real-time monitoring setup. The monitoring node is a mini-PC equipped with an AMD Ryzen 7 5800H processor (8 cores and 16 threads, base frequency 3.20 GHz), 32 GB DDR4 memory, and a 1 TB NVMe SSD. This device serves as the edge-side monitor and is responsible for executing packet capture, feature extraction, and real-time inference. The traffic-generating device (Net A) is a laptop equipped with a 13th-generation Intel Core i7-1365U CPU (10 cores and 12 threads, base frequency 1.80 GHz), 16 GB memory, and a

TABLE I
LIST OF THE 28 STATISTICAL FEATURES USED IN THE PROPOSED NTC SYSTEM.

Feature	Description
duration	Time span of packets within the window.
src_bytes	Total bytes of packets labeled as TCP.
dst_bytes	Total bytes of packets labeled as UDP.
pkt_count	Number of packets in the window.
pkt_rate	Packet arrival rate (packets per second).
mean_pkt_len	Mean packet length.
std_pkt_len	Standard deviation of packet length.
min_pkt_len	Minimum packet length.
max_pkt_len	Maximum packet length.
count	Same as pkt_count (included for compatibility).
srv_count	Number of distinct destination ports.
error_rate	Fraction of packets with SYN error flags.
srv_error_rate	Service-level SYN error rate.
error_rate	Fraction of packets with RST error flags.
srv_error_rate	Service-level RST error rate.
same_srv_rate	Proportion of packets using the same service.
diff_srv_rate	Proportion of packets using different services.
srv_diff_host_rate	Service switching across destination hosts.
dst_host_count	Number of distinct destination IPs.
dst_host_srv_count	Number of distinct (IP, port) destination pairs.
dst_host_same_srv_rate	Hosts sharing the same service.
dst_host_diff_srv_rate	Hosts using different services.
dst_host_same_src_port_rate	Hosts sharing the same source port.
dst_host_srv_diff_host_rate	Service-host diversity ratio.
dst_host_error_rate	SYN error rate at destination-host level.
dst_host_srv_error_rate	Service-level SYN error rate at hosts.
dst_host_rerror_rate	RST error rate at destination-host level.
dst_host_srv_rerror_rate	Service-level RST error rate at hosts.

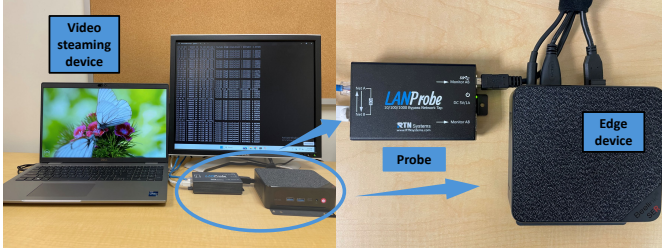


Fig. 1. Overall experimental environment used for real-time traffic monitoring and classification. The setup consists of a mini-PC acting as the edge-side monitor and a LANProbe passive network tap that mirrors bidirectional packets without disrupting the data path. A user laptop generates normal application traffic, while the monitor receives a complete copy of flows through the tap and runs the capture, feature extraction, and classification pipeline in real time. This configuration simulates a lightweight edge-monitoring device passively observing user network activities in a realistic deployment scenario.

512 GB KIOXIA BG6 NVMe SSD. The network tap used in the setup is a LANProbe 10/100/1000 Mbps bypass TAP, which mirrors bidirectional traffic without introducing any interruption or modification. Together, these components form a lightweight yet realistic platform for evaluating our real-time traffic classification pipeline. Based on this hardware, we construct an edge-side monitoring scenario in which the mini-PC is connected to the monitor port of the TAP and receives a complete copy of all bidirectional packets. The LANProbe

bridges the traffic between the user laptop and the upstream Ethernet network, ensuring that all application flows generated on Net A are fully observable at the monitor. The mini-PC continuously runs the real-time processing pipeline: captured packets are first obtained from the monitoring interface, then segmented into fixed-length 20-second observation windows, and finally converted into 28-dimensional statistical feature vectors for similarity computation and classification. During the experiments, the monitor is connected to an external display for program execution, debugging, and live logging, while the traffic-generating laptop runs common applications such as HD video streaming to produce realistic traffic patterns. This physical and logical configuration enables the monitor to passively observe user traffic and perform real-time feature extraction and classification without interfering with normal user activities.

C. Evaluation Metrics

1) *Classification Performance*: To evaluate the performance of the proposed NTC system, we use standard classification metrics including Precision, Recall, F1 Score, and Accuracy, defined as:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad (3)$$

$$\text{Recall} = \frac{TP}{TP + FN}, \quad (4)$$

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}, \quad (5)$$

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \quad (6)$$

Here, TP denotes windows that originate from a given application and are correctly identified. TN represents windows from other applications that are correctly rejected. FP refers to windows that belong to other applications but are incorrectly labeled as the target application. FN denotes windows from the target application that are incorrectly assigned to other classes.

2) *Distance-Based Evaluation*: Since the NTC system is driven by feature similarity rather than a deep neural classifier, we further evaluate the distance relationships between applications. Our primary metric is the proposed PF-NMSE, which measures per-feature normalized deviation between two windows. For comparison, we also implement two commonly used metrics, .e.g, Euclidean distance [24] and classical NMSE [25]. These metrics allow us to quantify how well each application's feature distribution separates from others, and to validate the effectiveness of PF-NMSE as the main decision criterion in our system.

D. Evaluation result of the proposed PF-NMSE

Fig. 3 presents the pairwise distance matrices computed using three similarity measures, i.e., Euclidean distance, classical NMSE, and the proposed PF-NMSE. Note that all values are column-wise min-max normalized within each class to highlight the relative separation strength among different

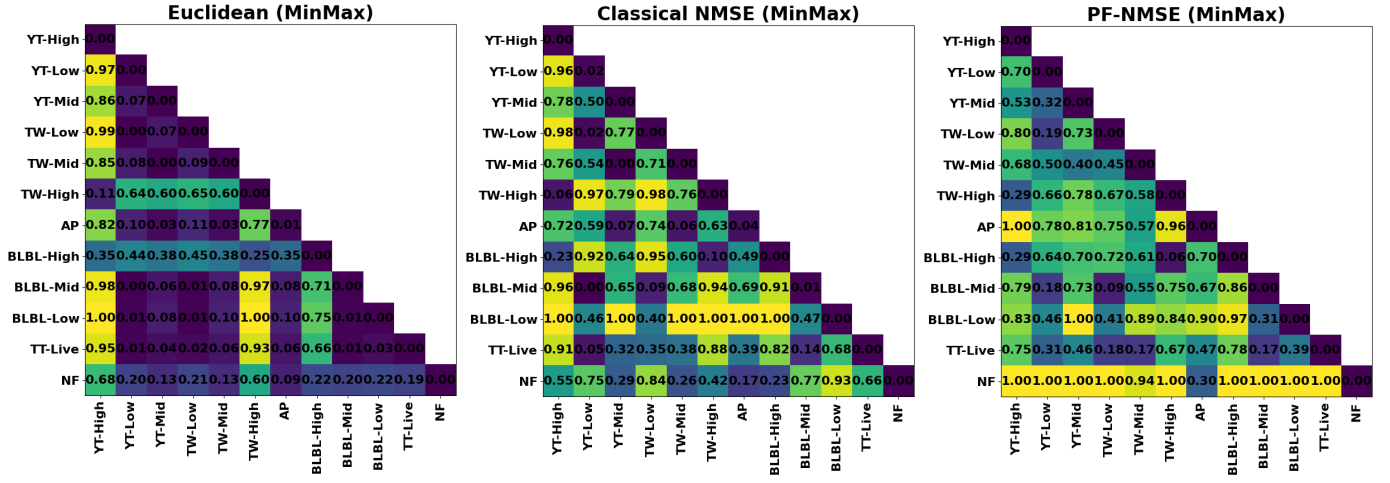


Fig. 2. Pairwise Distance Matrix across all application classes using three distance metrics: PF-NMSE, classical NMSE, and Euclidean distance. Each matrix visualizes the pairwise distance relationships among all applications and resolutions in the feature space. For all metrics, a larger distance(lighter) indicates greater dissimilarity between two application patterns, whereas a smaller distance suggests higher similarity. Ideally, distinct applications should exhibit larger pairwise distances to ensure clearer separability in the feature space.

application pairs. This normalization facilitates direct comparison of how each metric spreads inter-class distances and reveals whether certain traffic categories appear artificially close after scaling. Under the Euclidean metric, many traffic categories exhibit only minor differences, making it difficult to distinguish distinct patterns. For example, YT-High shows a normalized distance of 0.07 to YT-Mid and 0.11 to TW-High, while TW-Low and TW-Mid have distances as low as 0.07 and 0.08 despite representing different quality levels. Even cross-application pairs such as BLBL-Mid versus YT-Mid yield values around 0.07, indicating that Euclidean distance compresses multiple categories into a narrow numerical range and fails to provide reliable separability. Classical NMSE partially alleviates this issue by amplifying some cross-application differences; however, misleading similarities persist. For instance, YT-Low and TT-Live exhibit a distance of only 0.05, and TW-Mid and TW-Low remain closely clustered with distances around 0.08, even though their traffic dynamics differ. Some unrelated categories also appear nearly indistinguishable, such as BLBL-Low and AP, whose distance is only 0.01, suggesting that NMSE remains sensitive to dominant feature magnitudes. In contrast, the proposed PF-NMSE yields a more structured and discriminative separation. Distances between unrelated applications consistently shift toward higher values, e.g., AP versus YT-Low at 0.78, BLBL-Low versus AP at 0.89, and NF versus TT-Live at 0.30. Meanwhile, within-platform variants remain reasonably close yet clearly separable, as illustrated by YT-High, YT-Mid, and YT-Low, whose distances fall between 0.32 and 0.70 rather than collapsing toward zero as observed under Euclidean distance. PF-NMSE also effectively suppresses misleading similarities: pairs that were nearly indistinguishable under Euclidean distance and NMSE, such as BLBL-Mid versus YT-Mid, increase substantially to 0.73.

E. Evaluation result of classification performance

Table II summarizes the classification performance under different similarity thresholds τ for both offline and online evaluation settings. The offline setting refers to experiments conducted using collected data that are split into a feature library and a held-out local test set. The online setting corresponds to real-time testing while streaming videos, where predictions are generated continuously based on live traffic. Across both settings, a larger τ retains more samples but enforces weaker filtering, while a smaller τ produces stricter filtering and therefore higher drop rates.

In the offline evaluation, $\tau = 5$ achieves stable and balanced results, with average precision, recall, and F1-score all around 97%, and a very low average drop rate of 0.33%. When τ is reduced to 0.5, the drop rate increases to 4.55%, but the classification performance remains strong, with accuracy averaging 98.89%. At the strictest threshold $\tau = 0.05$, the average drop rate rises sharply to 27.64%, yet the remaining samples are classified with near-perfect accuracy (avg. accuracy 99.38%, avg. F1-score 99.34%). In the online evaluation, a similar trend is observed. With $\tau = 5$, the average accuracy is 95.02% and the drop rate remains low (0.30%). When τ is tightened to 0.5, the average accuracy increases to 97.56% with a moderate drop rate of 11.98%. At $\tau = 0.05$, the system filters out nearly half of the samples (avg. drop rate 49.09%), but the retained samples achieve the highest accuracy of 98.54% on average.

Fig. 3 shows the normalized confusion matrices under the same similarity threshold τ equals to 0.5 for both the offline evaluation and the online evaluation. The offline results display a very clear diagonal structure, indicating nearly perfect classification across all categories.

TABLE II
OFFLINE AND ONLINE CLASSIFICATION PERFORMANCE UNDER DIFFERENT SIMILARITY THRESHOLDS τ .

τ	Precision (%)			Recall (%)			Accuracy (%)			F1-score (%)			Drop rate(%)		
	min.	avg.	max.	min.	avg.	max.	min.	avg.	max.	min.	avg.	max.	min.	avg.	max.
Offline performance															
5	95.40	97.09	99.26	95.27	97.05	99.38	95.62	97.02	99.27	95.30	97.00	99.30	0.00	0.33	0.66
0.5	98.20	98.92	100.00	98.20	98.91	100.00	98.45	98.89	100.00	98.20	98.90	100.00	3.80	4.55	5.21
0.05	98.61	99.48	100.00	98.41	99.27	100.00	98.89	99.38	100.00	98.61	99.34	100.00	25.10	27.64	30.40
Online performance															
5	93.33	95.29	96.76	93.11	94.98	96.48	93.13	95.02	96.49	93.17	95.01	96.55	0.17	0.30	0.50
0.5	96.21	97.69	98.79	95.82	97.31	98.15	96.02	97.56	98.47	95.97	97.45	98.44	10.00	11.98	14.00
0.05	98.04	98.52	98.89	94.54	95.52	96.71	97.98	98.54	99.05	95.57	96.45	97.43	47.00	49.09	50.83

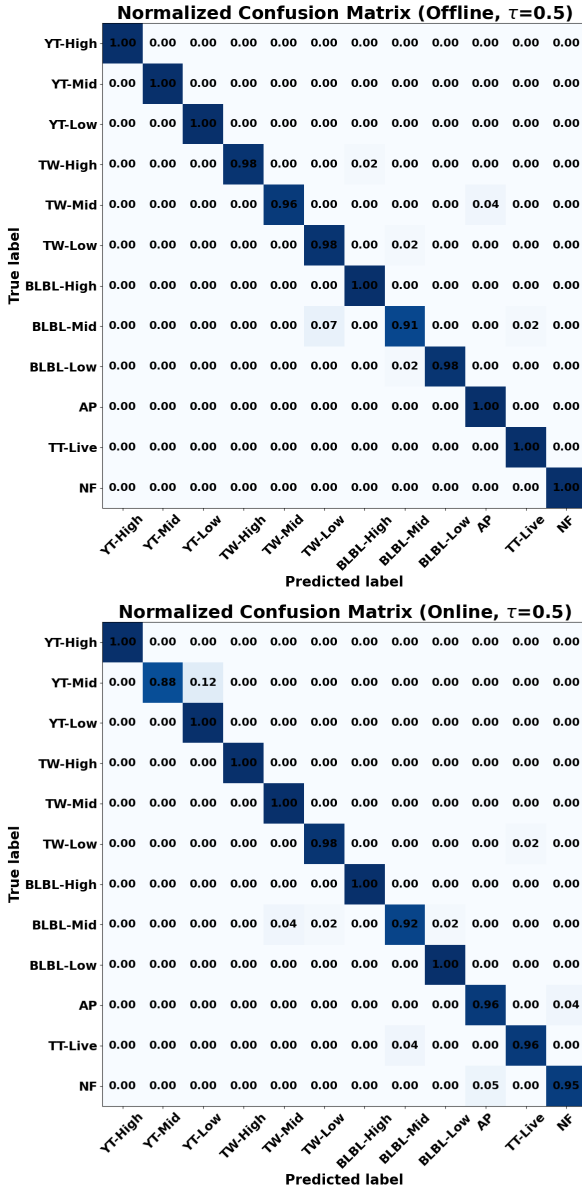


Fig. 3. Normalized confusion matrices for offline and online evaluations under the similarity threshold $\tau = 0.5$. The offline results are obtained from locally collected and preprocessed traffic with fixed feature libraries, while the online results are generated in real time during actual video streaming.

Most classes achieve a diagonal value of 1.00, including YT-High, YT-Mid, YT-Low, TW-High, TW-Mid, TW-Low, BLBL-High, BLBL-Mid, and NF, with almost no cross-class errors. Only a few closely related categories show minor deviations; for example, 0.07 of BLBL-Mid samples are misclassified as BLBL-Low, but the overall stability remains exceptionally high. In comparison, the online results maintain strong consistency but exhibit slight confusion between certain categories. For instance, the diagonal value of YT-Mid decreases from 1.00 in offline testing to 0.88 online, with 0.12 of its samples misclassified as YT-Low. TT-Live also shows a diagonal value of 0.96 online, with a small portion of samples incorrectly assigned to NF. Despite the dynamic nature of real network conditions, most categories retain accuracy comparable to their offline performance. Classes such as TW-High, TW-Mid, TW-Low, and BLBL-Low still achieve diagonal values between 0.98 and 1.00 in the online matrix, demonstrating the robustness of the method across real-world environments.

IV. DISCUSSION

In this paper, we develop a deployable platform for live network traffic classification that analyzes real Ethernet streams using lightweight statistical features and a PF-NMSE-based similarity measure. The system builds a reference feature library from controlled single-application sessions and performs online matching with a confidence threshold, enabling reliable identification of major video applications in real local area networks environments. Through these evaluations, we observe that the PF-NMSE metric significantly improves stability across heterogeneous feature scales, particularly when different applications exhibit large variations in packet count, throughput, and timing statistics. The threshold mechanism further enhances reliability by filtering noisy or ambiguous windows, leading to more consistent predictions under dynamic bandwidth conditions. At the same time, the results also reveal sensitivity to window segmentation and certain edge cases where applications with similar traffic patterns (e.g., mid-resolution YouTube vs. high-bandwidth Twitch) may become less separable under heavy background traffic. Several directions remain open for future work. First, the current

design uses a fixed 20-second window; exploring shorter windows, adaptive segmentation, or multi-scale analysis may improve responsiveness. Second, while this study employs a lightweight machine-learning-based classifier, the framework can be extended to incorporate alternative models or deep-learning approaches to handle richer flow representations or encrypted payload patterns. Finally, future studies may evaluate resource consumption, multi-device scalability, and long-term stability to advance the system toward broader real-world deployment.

V. CONCLUSION

Network traffic classification is essential for understanding application behaviors and managing modern networks, yet many existing methods rely on curated datasets or controlled environments, leaving a gap in real-world deployability. In this paper, we developed a practical classification platform that operates directly on live Ethernet streams using lightweight statistical features and a PF-NMSE-based similarity measure with confidence thresholding. The system constructs a reference feature library from controlled single-application sessions and performs online inference without accessing packet payloads. Experiments conducted under real local area network conditions show that the proposed platform achieves consistent performance across both offline and online settings, with an average accuracy of approximately 97%. Future work will explore alternative window configurations, additional machine learning models, and lightweight deep learning methods to further enhance real-time classification capability and extend the platform to a broader range of applications.

ACKNOWLEDGMENT

This work was partially supported by National Science Foundation under grants CNS-2451357 and OIA-2218046.

REFERENCES

- [1] A. O. Salau and M. M. Beyene, "Software defined networking based network traffic classification using machine learning techniques," *Scientific Reports*, vol. 14, no. 1, p. 20060, 2024.
- [2] I. A. Alwhbi, C. C. Zou, and R. N. Alharbi, "Encrypted network traffic analysis and classification utilizing machine learning," *Sensors*, vol. 24, no. 11, p. 3509, 2024.
- [3] J. Zhang, F. Li, F. Ye, and H. Wu, "Autonomous unknown-application filtering and labeling for dl-based traffic classifier update," in *IEEE INFOCOM 2020 - IEEE Conference on Computer Communications*, 2020, pp. 397–405.
- [4] R. H. Serag, M. S. Abdalzaher, H. A. E. A. Elsayed, M. Sobh, M. Krichen, and M. M. Salim, "Machine-learning-based traffic classification in software-defined networks," *Electronics*, vol. 13, no. 6, p. 1108, 2024.
- [5] R. M. AlZoman and M. J. Alenazi, "A comparative study of traffic classification techniques for smart city networks," *Sensors*, vol. 21, no. 14, p. 4677, 2021.
- [6] P. Lin, K. Ye, Y. Hu, Y. Lin, and C.-Z. Xu, "A novel multimodal deep learning framework for encrypted traffic classification," *IEEE/ACM Transactions on Networking*, vol. 31, no. 3, pp. 1369–1384, 2022.
- [7] Q. Xin, Z. Xu, L. Guo, F. Zhao, and B. Wu, "Iot traffic classification and anomaly detection method based on deep autoencoders," 2024.
- [8] A. Q. Mohammed and R. F. Ghani, "Network traffic classification to improve quality of service (qos)," in *AIP Conference Proceedings*, vol. 3282, no. 1. AIP Publishing LLC, 2025, p. 020007.
- [9] V. Tong, S. Souihi, H. A. Tran, and A. Mellouk, "Sdn-based application-aware segment routing for large-scale network," *IEEE Systems Journal*, vol. 16, no. 3, pp. 4401–4410, 2021.
- [10] J. Zhang, F. Li, H. Wu, and F. Ye, "Autonomous model update scheme for deep learning based network traffic classifiers," in *2019 IEEE Global Communications Conference (GLOBECOM)*, 2019, pp. 1–6.
- [11] S. Jebaraj and M. Joshi, "Port-based classification of network traffic analysis using the colasoftcapsa tool for enhanced cybersecurity," in *2023 International Conference on Communication, Security and Artificial Intelligence (ICCSAI)*. IEEE, 2023, pp. 1–5.
- [12] M. Çelebi, A. Özbilen, and U. Yavanoğlu, "A comprehensive survey on deep packet inspection for advanced network traffic analysis: issues and challenges," *Niğde Ömer Halisdemir Üniversitesi Mühendislik Bilimleri Dergisi*, vol. 12, no. 1, pp. 1–29, 2023.
- [13] L. Deri and F. Fusco, "Using deep packet inspection in cybertraffic analysis," in *2021 IEEE International Conference on Cyber Security and Resilience (CSR)*. IEEE, 2021, pp. 89–94.
- [14] J. Cao, D. Wang, Z. Qu, H. Sun, B. Li, and C.-L. Chen, "An improved network traffic classification model based on a support vector machine," *Symmetry*, vol. 12, no. 2, p. 301, 2020.
- [15] S. Bagui, X. Fang, E. Kalaimannan, S. C. Bagui, and J. Sheehan, "Comparison of machine-learning algorithms for classification of vpn network traffic flow using time-related features," *Journal of Cyber Security Technology*, vol. 1, no. 2, pp. 108–126, 2017.
- [16] Wei Wang, Ming Zhu, Xuewen Zeng, Xiaozhou Ye, and Yiqiang Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *2017 International Conference on Information Networking (ICOIN)*, 2017, pp. 712–717.
- [17] M. Lopez-Martin, B. Carro, A. Sanchez-Esguevillas, and J. Lloret, "Network traffic classifier with convolutional and recurrent neural networks for internet of things," *IEEE Access*, vol. 5, pp. 18 042–18 050, 2017.
- [18] L.-H. Chang, T.-H. Lee, H.-C. Chu, and C.-W. Su, "Application-based online traffic classification with deep learning models on sdn networks," 2019.
- [19] H. Yao, P. Gao, J. Wang, P. Zhang, C. Jiang, and Z. Han, "Capsule network assisted iot traffic classification mechanism for smart cities," *IEEE Internet of Things Journal*, vol. 6, no. 5, pp. 7515–7525, 2019.
- [20] F. Li and F. Ye, "Adaptive and lightweight network traffic classification for edge devices," *IEEE Transactions on Green Communications and Networking*, vol. 6, no. 4, pp. 2003–2014, 2022.
- [21] J. Zhang, F. Li, and F. Ye, "Network traffic clustering with qos-awareness," *China Communications*, vol. 19, no. 3, pp. 202–214, 2022.
- [22] A. Sharma and A. H. Lashkari, "A survey on encrypted network traffic: A comprehensive survey of identification/classification techniques, challenges, and future directions," *Computer Networks*, vol. 257, p. 110984, 2025.
- [23] J. Zhang, F. Li, and F. Ye, "Sustaining the high performance of ai-based network traffic classification models," *IEEE/ACM Transactions on Networking*, vol. 31, no. 2, pp. 816–827, 2023.
- [24] N. Krislock and H. Wolkowicz, "Euclidean distance matrices and applications," in *Handbook on semidefinite, conic and polynomial optimization*. Springer, 2012, pp. 879–914.
- [25] P. Händel, "Understanding normalized mean squared error in power amplifier linearization," *IEEE Microwave and Wireless Components Letters*, vol. 28, no. 11, pp. 1047–1049, 2018.